

CSC3002F - Operating Systems II

Scheduling Assignment

Connor Morrison (MRRCON006)

19th May 2026

Table of contents

Introduction	1
Methods	2
Method of data generation	2
Method of data validation	3
(a) Data completeness	3
(b) Code and data is logically correct	4
(c) Are the workloads identical across algorithms for the same seed	4
Results and Analysis	4
Per-Order metrics	4
Response time	4
Waiting time	4
Per-Patron metrics	4
Response and waiting time	5
Turnaround time	5
Throughput	5
Predictability	5
Fairness	5
Possibility of starvation	5
Conclusions and Recommendations	10

Introduction

Allegra the barman is an abstraction of the low-level workings of a CPU. The study of CPU scheduling algorithms can be likened much to the analogy of a barman serving drinks at a bar,

where each patron (customer) is a process and each of their drink orders is a CPU burst. This provides us with a real world example of what CPU scheduling algorithms do and what these algorithms aim to achieve.

The CPU scheduling algorithms that will be tested within this analogy are First-Come First-Serve (FCFS), Shortest Job First (SJF), Priority, and Multilevel Feedback Queue (MLFQ). We are provided the information that none of the algorithms use pre-emption at the level of the individual drink order.

The goal of this assignment is to determine the CPU scheduling algorithm that, when applied to the bar setting, maximises satisfaction for the most patrons.

We will evaluate the overall success of the algorithms by comparing their performances with respect to the response and waiting times (both per order, and the total per patron), and turnaround time, as well as throughput, predictability, fairness, and possibility of starvation.

We need to decide, in the context of a bar setting, which of their evaluation metrics are the most important for a bar owner.

Methods

One of the most important steps in the experimental process is creating valid and reliable data in a repeatable way. This process can be broken down into steps to first generate the data, and second, validate the data.

```
for patrons in 5 20 50
do
  for seed in {1..30}
  do
    for schedalg in 0 1 2 3
    do
      #args format ARGS="patrons scheduling_algorithm 0 seed"
      make --silent run ARGS="$patrons $schedalg 0 $seed" 2>&1 >/dev/null | sed "s/~/\$/patrons,$schedalg,$seed,/" >> simulation_results.csv
    done
  done
done
```

Figure 1: Optional Caption Text

Method of data generation

We need to guarantee that every algorithm gets tested in the exact same circumstances. To ensure the reliability of the results, we must include multiple repetitions for each (total_patrons, scheduling_algorithm) combination. For the assumption of normality, according to the central limit theorem, it is a minimum requirement to have 30 observations per scenario. The java code given ensures that for some seed, the number of orders and the arrival time of the patrons remains constant. So we can loop from seed (1 to 30) to guaranteed unique

To ensure that we fairly tested each algorithm

To generate this data for the analysis of the scheduling algorithms, the following steps were taken.

- Complete the `recordCompletedOrder` function in `Barman.java` to print all of the per order metrics that the `barman` class provided in the format “patron,drink,priority,queue_level,seq_no,service_start_time,arrival_time,completion_time,waiting_time,imbibing_time,turnaround_time,enqueue_time,execution_time,response_time”
- Create a bash script that loops through

Method of data validation

To validate the data and code for the analysis of the algorithms, we need to ensure that

(a) the data is complete, (b) the data follows the basic CPU scheduling rules, and (c) the data produced identical workloads for each algorithm across seeds.

(a) Data completeness

We check that the correct amount of simulations was run and that there were no missing values

Validation: Seeds per workload/algorithm combination:

total_patrons	schedalg	seeds_count
5	FCFS	30
5	SJF	30
5	Priority	30
5	MLFQ	30
20	FCFS	30
20	SJF	30
20	Priority	30
20	MLFQ	30
50	FCFS	30
50	SJF	30
50	Priority	30
50	MLFQ	30

Number of missing values: 0

Which correctly matches what we expect to see.

(b) Code and data is logically correct

To test this we check whether all $\text{Turnaround} \geq \text{Waiting} + \text{Execution}$, and $\text{Response} \leq \text{Waiting}$. If there are 0 logic errors, then that shows that the code and data generated is valid.

0 exceptions, Validation Passed: Turnaround times are logically consistent.

0 exceptions, Validation Passed: Response time is always $\leq$ total waiting time.

(c) Are the workloads identical across algorithms for the same seed

We tested whether the each algorithm had and identical total and type of drinks for the same seed, total_patrons pairing.

Passed! The workload produced was identical across algorithms for a given seed

Results and Analysis

Per-Order metrics

What does this mean in terms of the bar and CPU scheduling

Response time

Waiting time

Per-Patron metrics

What does this mean in terms of the bar and CPU scheduling

Response and waiting time

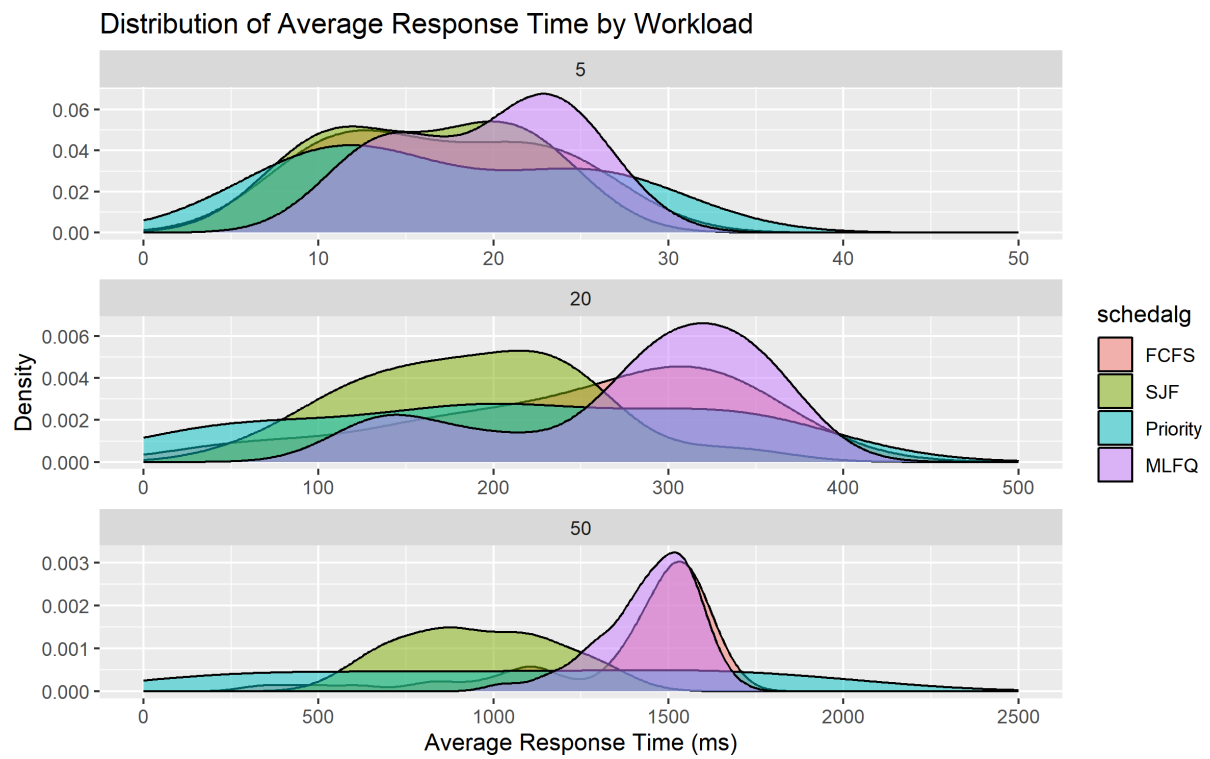
Turnaround time

Throughput

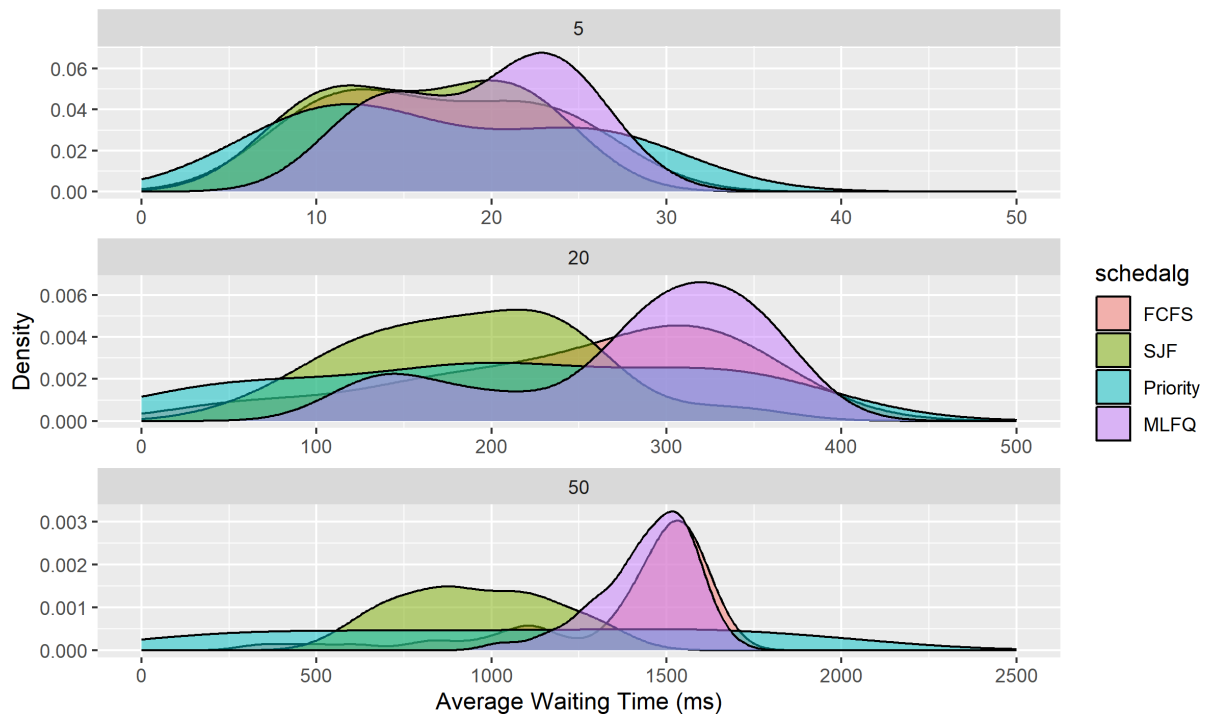
Predictability

Fairness

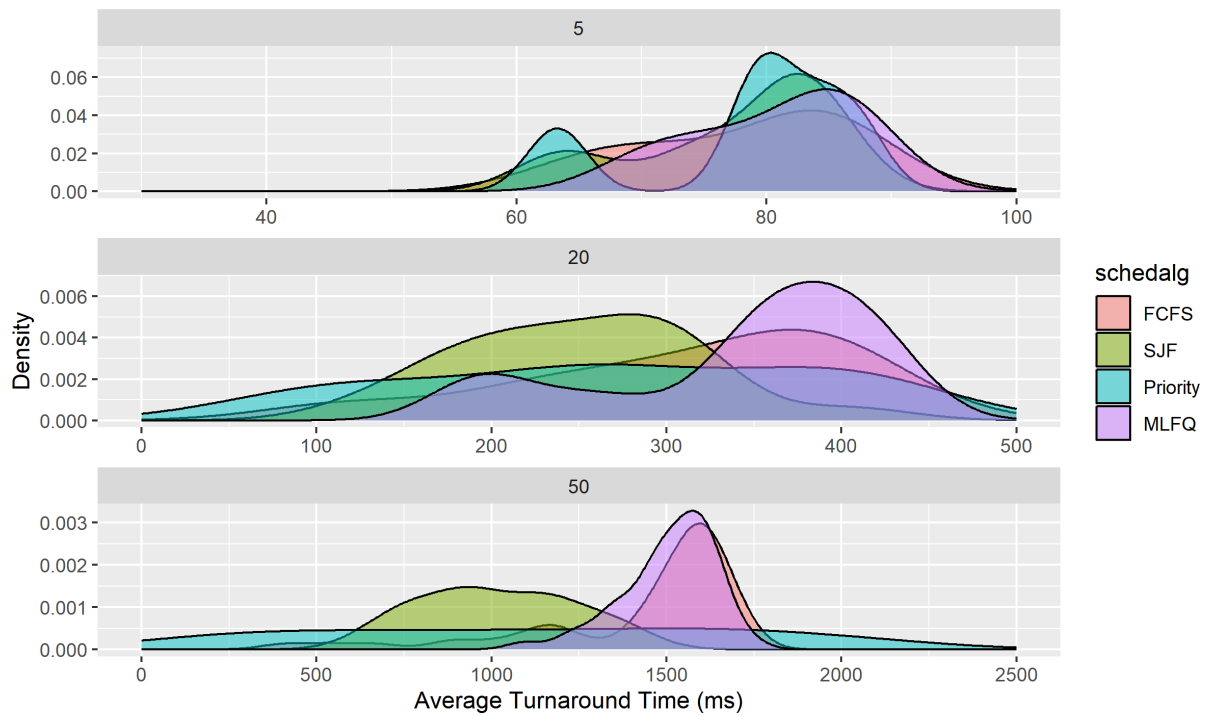
Possibility of starvation



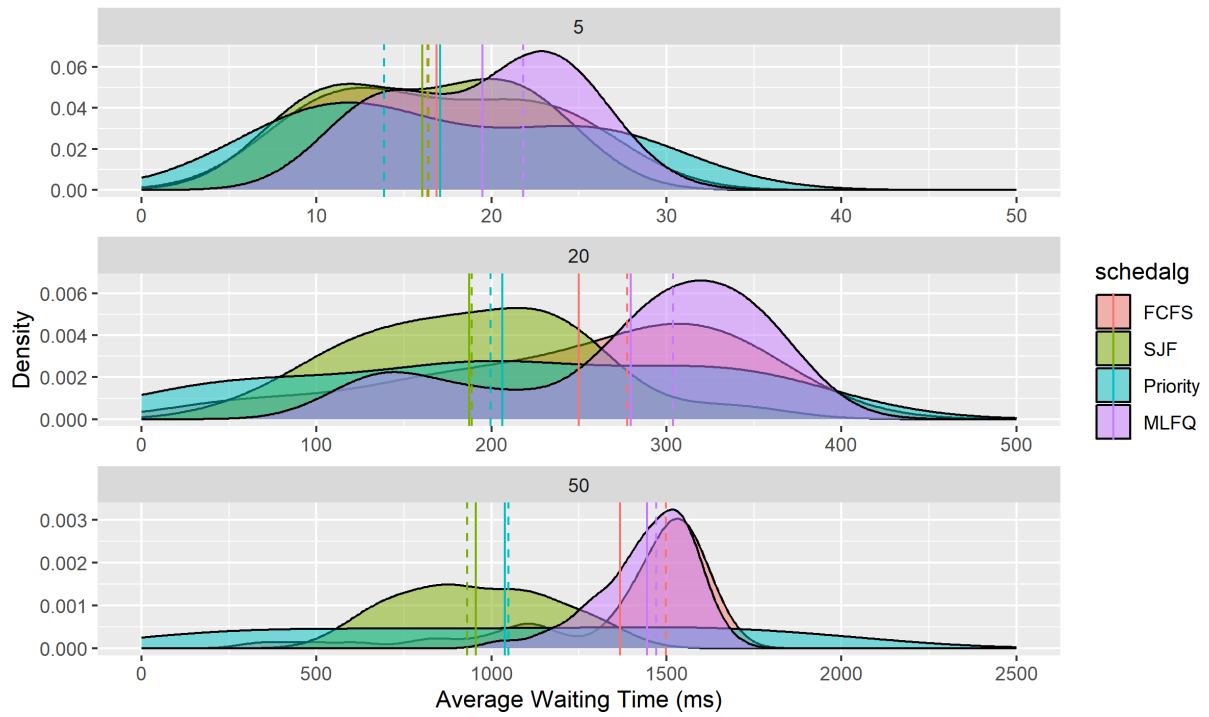
Distribution of Average Waiting Time by Workload



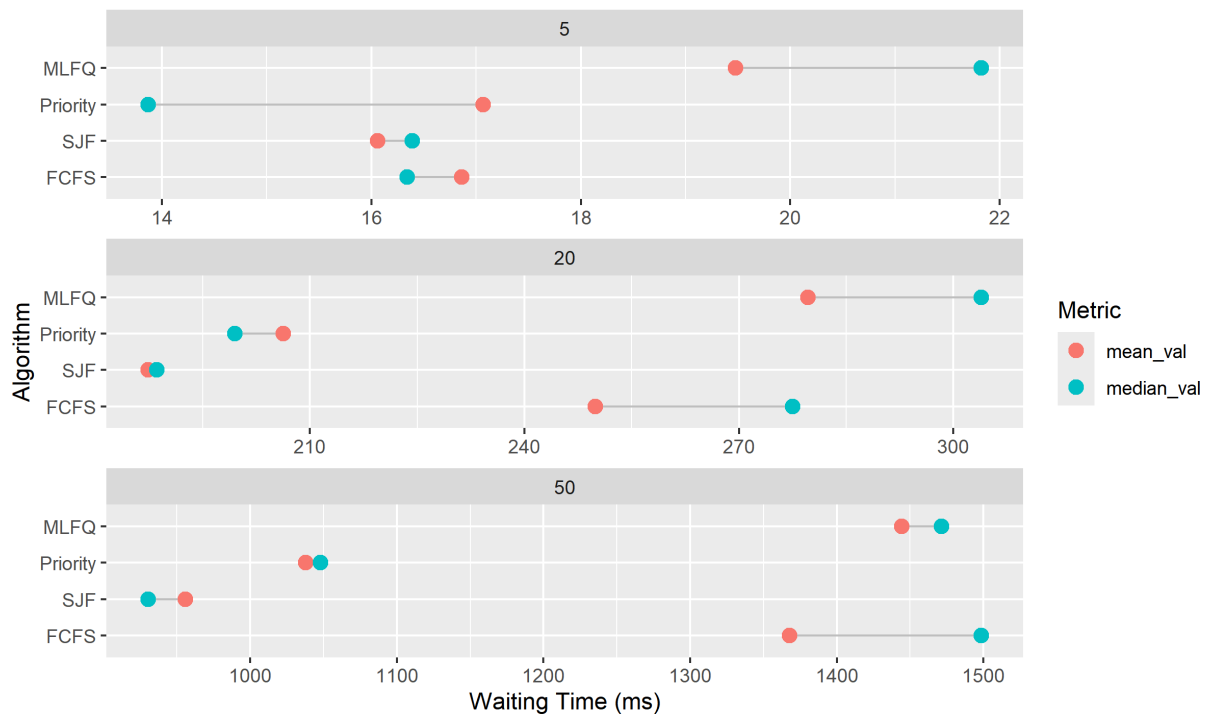
Distribution of Average Turnaround Time by Workload



Waiting Time Distributions with Mean (Solid) and Median (Dashed)

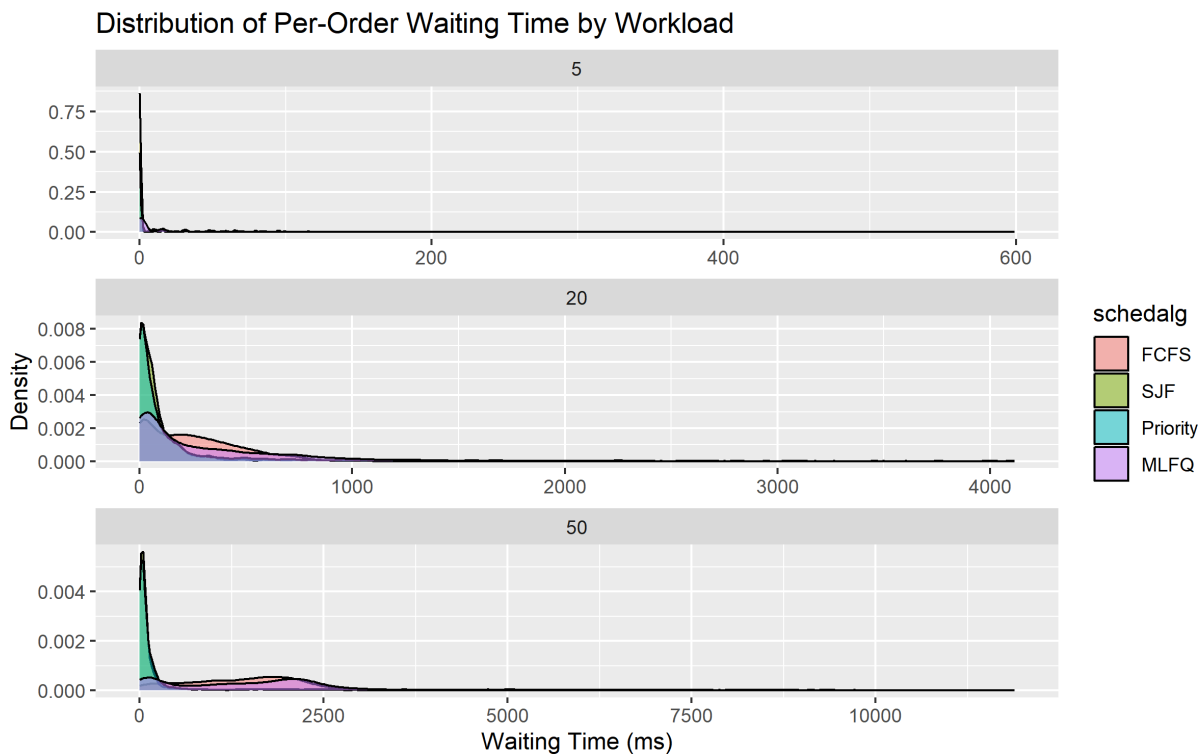


Mean vs Median Wait Time Gap by Algorithm



```
ggplot(df, aes(x = waiting_time, fill = schedalg)) +
  geom_density(alpha = 0.5) +
  facet_wrap(~total_patrons, ncol = 1, scales = "free") +

  labs(title = "Distribution of Per-Order Waiting Time by Workload",
       x = "Waiting Time (ms)",
       y = "Density")
```

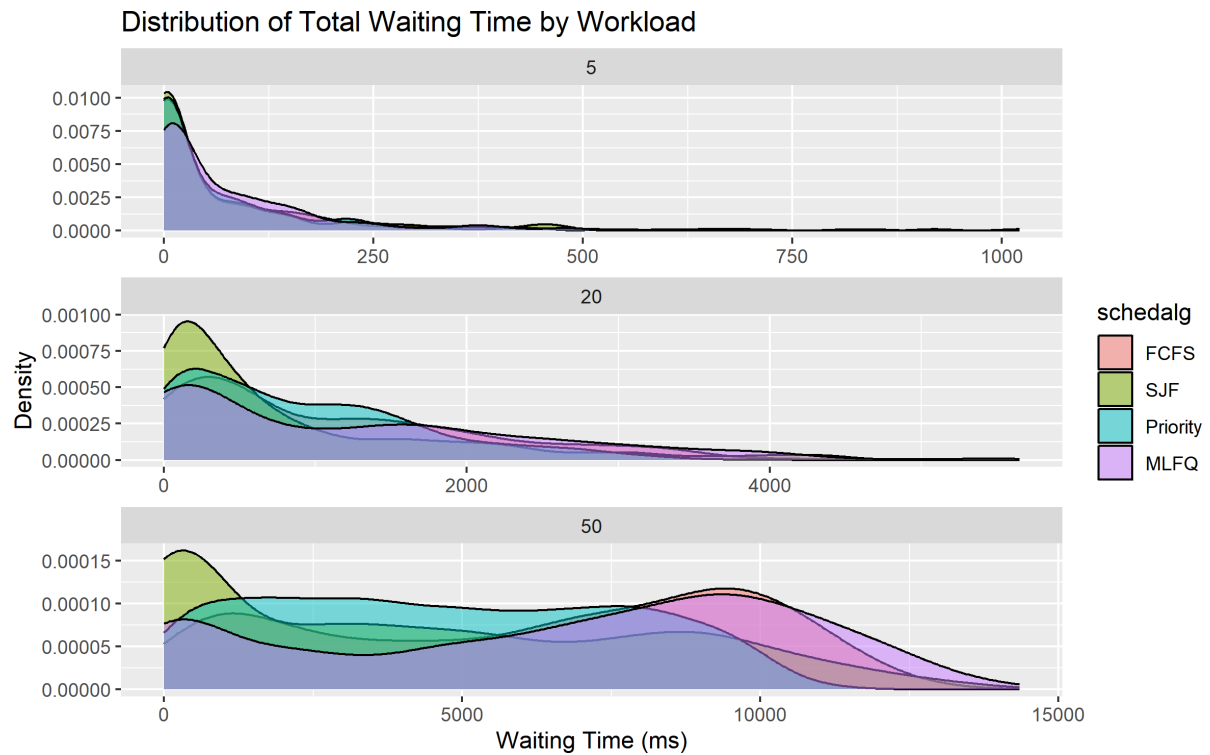


```
patron_df <- df %>%
  group_by(total_patrons, schedalg, seed, patron) %>%
  summarise(
    total_wait = sum(waiting_time),
    turnaround = max(completion_time) - min(arrival_time),
    num_drinks = n(),
    .groups = 'drop'
  )

ggplot(patron_df, aes(x = total_wait, fill = schedalg)) +
  geom_density(alpha = 0.5) +
  facet_wrap(~total_patrons, ncol = 1, scales = "free") +
```

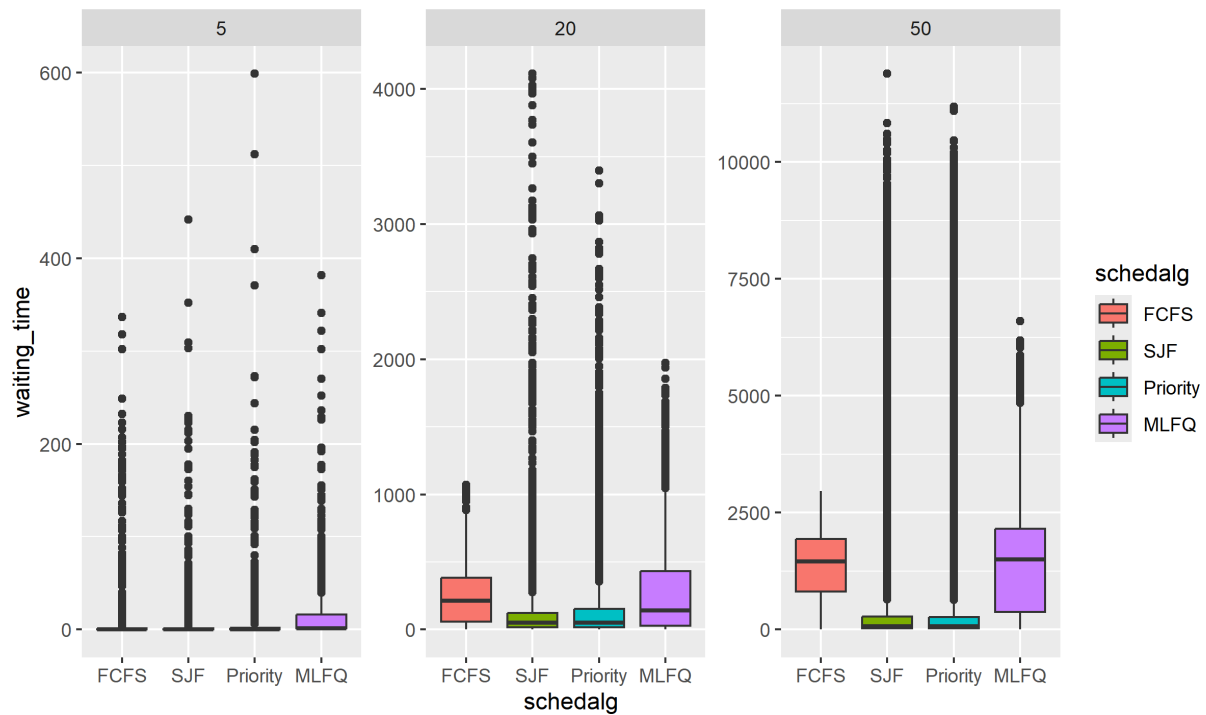


```
labs(title = "Distribution of Total Waiting Time by Workload",
     x = "Waiting Time (ms)",
     y = "Density")
```



```
ggplot(df, aes(x=schedalg, y=waiting_time, fill=schedalg)) +
  geom_boxplot() +
  facet_wrap(~total_patrons, scales = "free") +
  labs(title="Waiting Time Distribution by Algorithm")
```

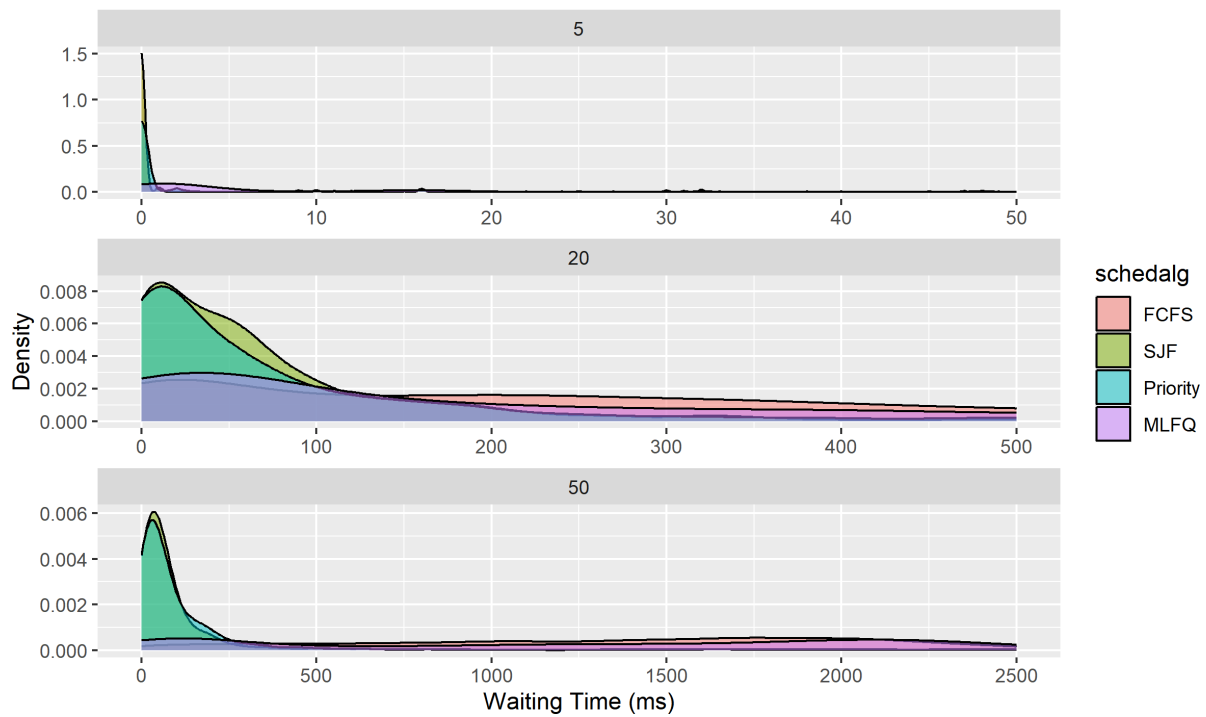
Waiting Time Distribution by Algorithm



Conclusions and Recommendations

```
# Note: Using the raw 'df' to capture every single drink order
ggplot(df, aes(x = waiting_time, fill = schedalg)) +
  geom_density(alpha = 0.5) +
  facet_wrap(~total_patrons, ncol = 1, scales = "free") +
  faceted_pos_scales(
    x = list(
      total_patrons == 5 ~ scale_x_continuous(limits = c(0, 50)),
      total_patrons == 20 ~ scale_x_continuous(limits = c(0, 500)),
      total_patrons == 50 ~ scale_x_continuous(limits = c(0, 2500))
    )
  ) +
  labs(title = "Distribution of Individual Order Waiting Times",
       x = "Waiting Time (ms)", y = "Density")
```

Distribution of Individual Order Waiting Times



```
# 1. Summarize by Patron
patron_summary <- df %>%
  group_by(total_patrons, schedalg, patron) %>%
  summarise(total_wait = sum(waiting_time), .groups = 'drop') %>%
  group_by(total_patrons, schedalg) %>%
  summarise(Mean = mean(total_wait), Median = median(total_wait), .groups = 'drop') %>%
  pivot_longer(cols = c(Mean, Median), names_to = "Metric", values_to = "Value")

# 2. Plot
ggplot(patron_summary, aes(x = schedalg, y = Value, fill = Metric)) +
  geom_col(position = "dodge") +
  facet_wrap(~total_patrons, scales = "free_y", ncol = 1) +
  labs(title = "Average vs Median Total Wait per Patron",
       subtitle = "A large gap between Mean and Median indicates Starvation",
       y = "Time (ms)", x = "Algorithm")
```

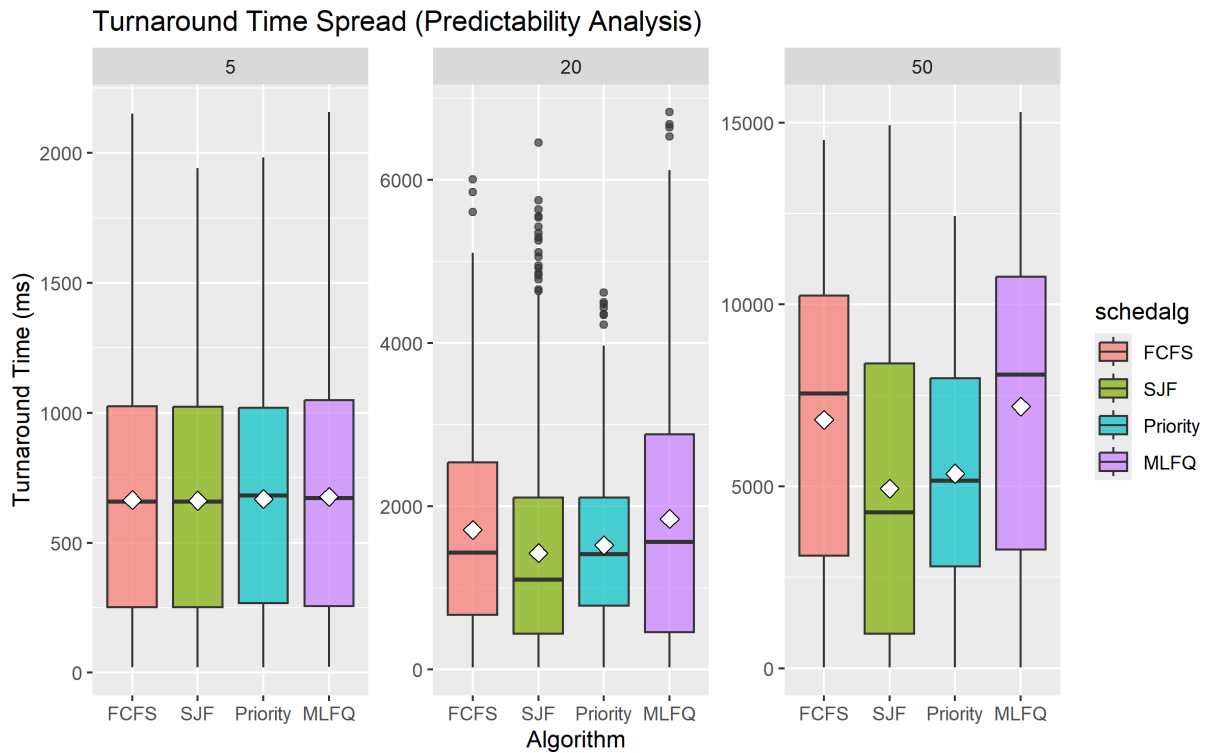
Average vs Median Total Wait per Patron

A large gap between Mean and Median indicates Starvation



```
# Using the patron-level data to see the spread of Turnaround Times
patron_df <- df %>%
  group_by(total_patrons, schedalg, seed, patron) %>%
  summarise(turnaround = max(completion_time) - min(arrival_time), .groups = 'drop')

ggplot(patron_df, aes(x = schedalg, y = turnaround, fill = schedalg)) +
  geom_boxplot(alpha = 0.7) +
  stat_summary(fun = mean, geom = "point", shape = 23, size = 3, fill = "white") +
  facet_wrap(~total_patrons, scales = "free_y") +
  labs(title = "Turnaround Time Spread (Predictability Analysis)",
       y = "Turnaround Time (ms)", x = "Algorithm")
```

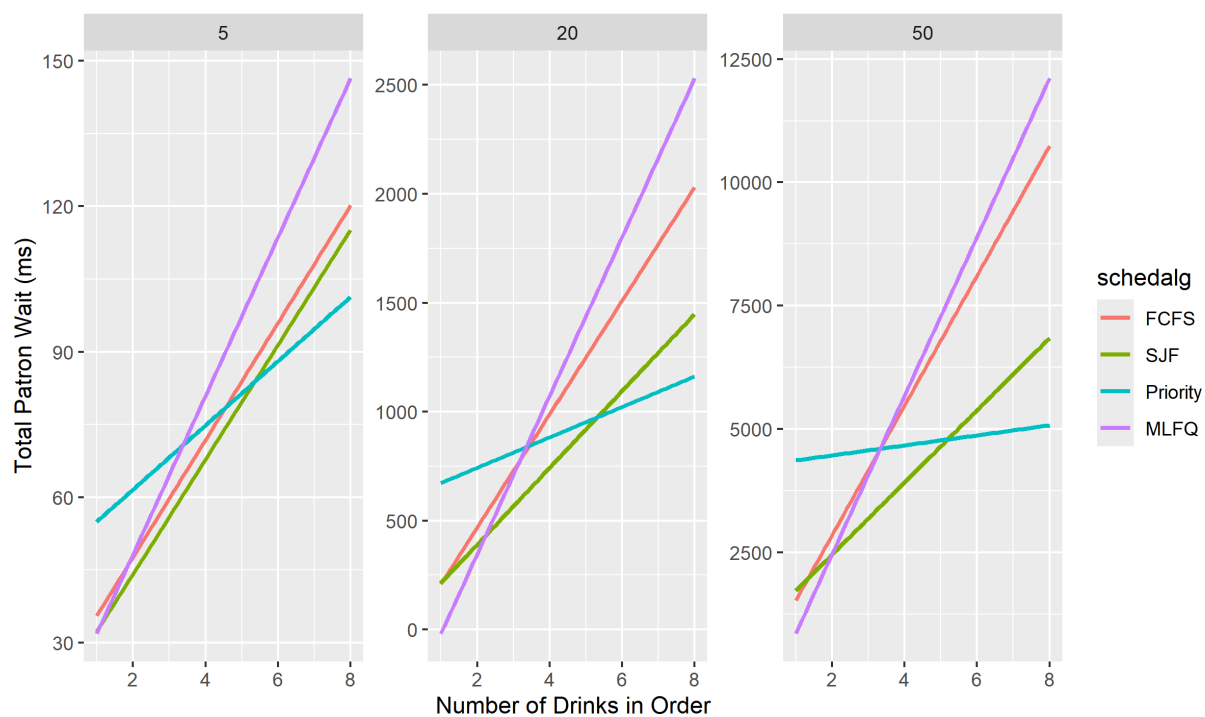


```
# Calculate total wait and order size per patron
fairness_df <- df %>%
  group_by(total_patrons, schedalg, seed, patron) %>%
  summarise(total_wait = sum(waiting_time),
            order_size = n(), .groups = 'drop')

ggplot(fairness_df, aes(x = order_size, y = total_wait, color = schedalg)) +
  #geom_jitter(alpha = 0.5) + # Jitter helps see overlapping points
  geom_smooth(method = "lm", se = FALSE) + # Adds a trend line
  facet_wrap(~total_patrons, scales = "free") +
  labs(title = "Fairness: Total Wait Time vs. Order Size",
       x = "Number of Drinks in Order", y = "Total Patron Wait (ms)")
```

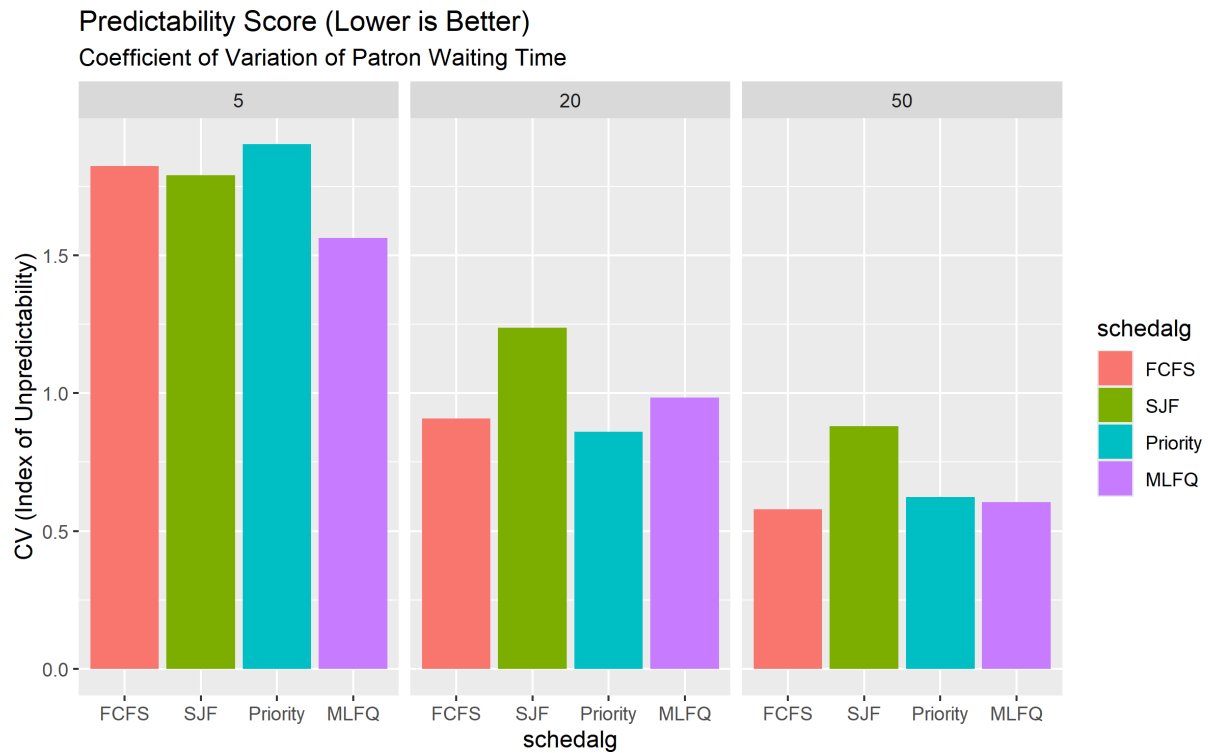
`geom\_smooth()` using formula = 'y ~ x'

Fairness: Total Wait Time vs. Order Size



```
predictability_df <- fairness_df %>%
  group_by(total_patrons, schedalg) %>%
  summarise(CV = sd(total_wait) / mean(total_wait), .groups = 'drop')

ggplot(predictability_df, aes(x = schedalg, y = CV, fill = schedalg)) +
  geom_col() +
  facet_wrap(~total_patrons) +
  labs(title = "Predictability Score (Lower is Better)",
        subtitle = "Coefficient of Variation of Patron Waiting Time",
        y = "CV (Index of Unpredictability)")
```



```
# 1. Calculate the final finish time for every simulation run
throughput_df <- df %>%
  group_by(total_patrons, schedalg) %>%
  summarise(total_time = max(completion_time), .groups = 'drop')

# 2. Plot the average "Closing Time" for the bar
ggplot(throughput_df, aes(x = schedalg, y = total_time, fill = schedalg)) +
  stat_summary(fun = mean, geom = "col") +
  facet_wrap(~total_patrons, scales = "free_y", ncol = 1) +
  labs(title = "Bar Efficiency: Total Time to Serve All Patrons",
       subtitle = "Lower time indicates higher throughput",
       x = "Algorithm", y = "Final Completion Time (ms)")
```

Bar Efficiency: Total Time to Serve All Patrons

Lower time indicates higher throughput

